Agile.NET Static Method Decryption

Reverse-engineering case study | de4dotEx | AgileNetMode | October 2026

EXECUTIVE SUMMARY

The investigation recovered the static managed method bodies of Agile.NET-protected NinjaTrader.Core.dll (x64). The original Decrypter5 transformation failed on RID 4. A sequence of controlled diagnostics reconstructed a 16-byte cyclic XOR keystream from redundant V52 metadata. Every one of 10,328 nonempty indexed records subsequently passed MethodBodyParser's structural checks.

The user independently observed approximately 80-90% of the assembly as readable decompiled C#. The final implementation derives the keystream at runtime instead of hardcoding assembly-specific bytes. Existing dynamic fallback is retained.

SCOPE AND PROTECTION ARCHITECTURE

Target: NinjaTrader.Core.dll, managed .NET x64 assembly.

Static protection: Agile.NET encrypted ordinary managed method bodies.

Separate virtualization: approximately 38 VM methods and 855 handlers in _CSVM.

Native runtime: AgileDotNetRT64.dll, protected by Themida.

Decision: focus on ordinary static method decryption, not VM devirtualization or Themida unpacking.

1. ORIGINAL FAILURE AND V52 HEADER VALIDATION

Header signature Normal; file offset 0x00340880; encrypted-code region size 0x0027BDD8.

MethodDef count 24,881; stored and actual MethodDef offset both 0x0004AA30; row size 18 bytes.

V52 index offset 0x005BC688; entry size 16 bytes; total index 398,096 bytes.

Index: 10,328 nonempty records, 14,553 absent; zero records outside the declared region.

RID 4: code offset 0x40, size 24, absolute file offset 0x003408C0. Legacy Decrypter5 produced first byte 0xAF, which cannot identify a valid tiny or fat CIL method header. InvalidMethodBody aborted static decryption and triggered x64-incompatible dynamic fallback.

2. DIAGNOSTIC EXPERIMENTS

A bounded, independent 512-record test was added so RID 4 would no longer prevent observing other encrypted methods.

Initial results: 143 structural parser successes, all tiny headers; zero valid fat headers. 90 fat-header-size failures, 230 invalid discriminators, and 49 tiny-size overruns.

Fat headers repeatedly produced transformed 0xDD33 instead of expected flags 0x3013. Local signature differences likewise showed stable XOR relationships.

Across 512 nonzero-flag records, observed mask differences fell into four groups rather than appearing random. This was consistent with a position-dependent periodic keystream.

Four alternative transforms were checked: raw, first XOR only, second XOR only, and legacy double XOR. None reconstructed valid fat headers or matched method metadata. Apparent tiny-header successes were not sufficient evidence of real CIL recovery.

3. RECOVERED 16-BYTE KEYSTREAM

Candidate cyclic keystream (phase relative to encrypted code region):

B4 BE 32 8D 00 81 E3 4D C4 59 4A 9E BC F7 7C E4

plaintext[j] = ciphertext[j] XOR mask[(methodInfo.codeOffs + j) & 15]

The mask was recovered from two known fat-header fields duplicated in cleartext V52 index metadata: method flags at header bytes 0-1 and local signature at bytes 8-11.

RID 5 encrypted prefix: D7 69 49 9E FD F7 7C E4 B5 BE 32 9C

RID 5 recovered prefix: 13 30 03 00 41 00 00 00 01 00 00 11

Decoded RID 5 fat header: flags 0x3013; maxStack 3; IL code size 65; local signature 0x11000001. Its flags and signature match the V52 index exactly.

The four repeating mask classes result from different record start offsets modulo the 16-byte period. This proves a consistent alignment rule for this assembly but does not independently prove the native runtime’s original key-generation mechanism.

4. FULL CANDIDATE-KEY VALIDATION

All 10,328 nonempty method records were tested independently after applying the recovered cyclic mask.

MethodBodyParser successes: 10,328 / 10,328 (100%).

Tiny structures: 4,874.

Fat structures without additional sections: 3,971.

Fat structures with additional sections: 1,483.

Fat-header flags matching index: 5,454 / 5,454.

Fat-header local signatures matching index: 5,454 / 5,454.

Invalid structural headers: 0. Read errors: 0.

These results provide very strong evidence of correct method-body structure recovery. MethodBodyParser does not validate every IL opcode, branch target, exception-handler semantic constraint, or runtime dependency.

5. FINAL IMPLEMENTATION

Modified file: de4dot.code/deobfuscators/Agile_NET/MethodsDecrypter.cs.

V52 processing derives every keystream byte using encrypted fat headers and corresponding metadata index flags and local signatures. Key material is aligned to (methodInfo.codeOffs + byte index) modulo 16.

Any conflicting or missing keystream phase rejects the inferred key. All nonempty records must pass structural parsing and fat-header metadata comparison before selecting the inferred-key Decrypter5 implementation.

If inference or verification fails, the original decryption implementation and dynamic fallback remain available.

The known candidate keystream is NOT hardcoded in the production decryption path. Diagnostic instrumentation remains in the branch and may later be placed behind a verbose diagnostics option.

6. SELECTED GITHUB MILESTONES

c98dbb5 / 33bb917 / 3fb8fbe - Log static decryption failures, RIDs, byte prefixes.

653a5d9 / 3cd6848 - Validate V52 header/index offsets and record bounds.

6be1d82 / 2aa8adc - Independent sample and categorized parser failures.

12ab924 / d8f7a018 - Cross-record XOR-mask diagnostics.

5bb2529 - Compare raw, key0, key7, and double-XOR variants.

af6fc03 - Full candidate key test, 10,328 / 10,328 structural success.

d7ef2cb8 - Infer, verify and use recovered V52 keystream in Decrypter5.

Repository: https://github.com/ex1dreamer/de4dotEx

Branch: AgileNetMode. Production checkpoint: d7ef2cb82aa1dc02dd02bbe797877b37355fb917.

7. OUTCOMES, LIMITATIONS, NEXT STEPS

Confirmed: V52 index consistency, 100% method-body structural parsing of nonempty records, 5,454/5,454 fat-header metadata matches, and user-reported 80-90% source readability.

Not established: verified IL instruction semantics for every method, successful rebuilding/loading of the whole assembly, devirtualization of VM-protected methods, or full native key-generation algorithm.

Next: checkpoint working branch, validate IL opcode streams and exception handlers, smoke-test decompilation and assembly behavior, then gate diagnostic logging.

Engineering insight: leveraging redundancy between encrypted method bodies and cleartext metadata avoided costly unpacking of the Themida-protected native runtime.

EVIDENCE BASIS

Prepared from the project handoff, the logged experiments and GitHub commits, and user confirmation of source availability. No new independent execution of NinjaTrader.Core.dll was performed for this report.